软件工程

实验二：软件设计与建模

活动票务与报名系统（票集）

陆知渔 241880173

2026 年 10 月 31 日

目录

一、软件的主要功能 3

1.1 系统概述 3

1.2 功能模块划分 3

1.3 模块之间的交互 4

二、软件架构 UML 图 5

2.1 结构性图 5

2.2 行为性图 8

2.3 交互性图 11

2.4 设计取舍 12

2.5 架构小结 13

三、UI 界面及功能描述 14

四、大模型的使用 18

4.1 使用原则 18

4.2 使用记录 18

4.3 工作流反思：能否只做检查校对 20

4.4 对大模型辅助软件设计的思考 20

五、Git 版本记录 22

5.1 提交与标签约定 22

六、总结 23

附录 A 提交文件结构 24

附录 B 图表生成方式 24

附录 C 需求追溯 24

一、软件的主要功能

1.1 系统概述

“票集”是面向校园与城市活动的票务与报名系统。参与者查询并筛选活动，选择场次与票种后报名或登记候补，报名成功后获得入场凭证；组织者维护本人活动、场次与票种，提交审核并查看名单、统计与导出；检票人员在被授权场次核验凭证并签到；平台管理员审核活动、取消违规活动，并查询平台汇总与操作日志。系统把名额一致性、并发安全、截止时间与关键操作可追溯作为设计重点，这些约束直接来自实验一的 29 条已确认需求。

角色	主要职责	对应需求
参与者	按条件查询活动；选择场次票种报名、登记候补或取消；查看本人凭证与签到状态。	REQ-007～REQ-015、REQ-017
组织者	维护本人活动、场次与票种；提交审核；查看报名名单、统计并导出 CSV。	REQ-002～REQ-004、REQ-022、REQ-023
检票人员	仅核验被授权场次的凭证，完成签到并写入操作日志。	REQ-016、REQ-017、REQ-030
平台管理员	审核或驳回活动、取消违规活动；查看平台汇总与操作日志。	REQ-003、REQ-006、REQ-024、REQ-025
定时任务（系统）	报名截止后使剩余候补失效；场次结束后将未签到的已报名记录标为缺席。	REQ-015、REQ-017

表 1-1 角色与职责

1.2 功能模块划分

依据用例与数据关系，系统划分为 8 个业务模块，另把“名额与事务”作为横切约束贯穿报名、取消、递补与检票四条写入路径。模块与实验一需求的对应关系如下表，未列入的 REQ-027 已合并至 REQ-023，REQ-029 为性能约束，在设计中体现为查询条件与索引策略，不单独成模块。

模块	主要功能	对应需求
M1 用户与权限	账号唯一性；四类角色；服务端角色与数据范围校验。	REQ-001、REQ-019、REQ-020、REQ-030
M2 活动与场次管理	活动、场次、票种维护；六态生命周期；容量下限约束。	REQ-002、REQ-004、REQ-005、REQ-019、REQ-021

模块	主要功能	对应需求
M3 审核与发布	提交审核；通过或驳回；重要信息复审；平台汇总。	REQ-003、REQ-004、REQ-024
M4 报名与候补	开放期报名与凭证；取消并释放名额；候补队列与自动递补；请求幂等。	REQ-008～REQ-015
M5 检票签到	场次授权校验；凭证核验；重复签到识别；缺席标记。	REQ-016～REQ-018
M6 统计与导出	容量、报名、候补与签到率；名单筛选；CSV 导出失败处理。	REQ-022、REQ-023、REQ-024
M7 通知	报名、递补与审核结果通知；失败记录与补发，不回滚业务。	REQ-026
M8 审计日志	记录关键操作的人员、时间、对象与结果，支持条件查询。	REQ-025
横切：名额与事务	容量 = 占用 + 剩余；报名、取消、递补、检票的事务化与并发复查。	REQ-021、REQ-028

表 1-2 功能模块与需求映射

1.3 模块之间的交互

模块之间保持单向依赖：表现层只调用应用服务，应用服务编排领域模型与基础设施。报名请求由报名服务统一入口，先经幂等与截止策略，再进入事务；活动与场次模型负责名额与状态不变量，候补管理器负责队列顺序，通知服务在事务提交后被调用。审核服务改变活动状态后，活动服务与报名服务在后续写入前都会复查活动状态，从而满足“取消后不得有并发报名成功”的要求。检票服务只读取报名记录并写入签到，统计与导出服务则以只读方式汇总名单。审计日志不单独被页面调用，而由各应用服务在关键操作中统一写入，失败操作同样留痕。

二、软件架构 UML 图

本章共给出 7 张 UML 图，覆盖实验要求的三类：结构性图包括类图、服务类图和组件图；行为性图包括用例图、活动图和状态图；交互性图用时序图表达。每张图先说明画了什么，再说明为什么这样设计，以及它对应哪些需求。读图约定如下：带 E 标记的是状态枚举，带 C 标记的是类；实线箭头表示关联或依赖，虚线箭头表示使用或通知；连线两端的数字与 * 表示多重性；黄色便签只写需求约束，不参与建模。

2.1 结构性图

2.1.1 类图

图 2-1 是领域类图，只保留业务概念，不含界面类与数据库实现。User 通过 Role 枚举区分四类角色；Event 用 EventState 的六种状态表达生命周期；EventSession 与 TicketType 分别记录容量与占用数，并通过 remaining() 统一给出剩余名额，避免“容量只挂在票种上、场次没有容量”的口径分裂。Registration 关联用户、场次与票种，用 RegistrationState 的七种状态替代布尔字段；WaitlistEntry 单独保存队列号与候补状态；Credential、CheckInRecord、OperationLog、Notification 分别承载凭证、签到事实、审计与通知。图中注释列出 REQ-011 明确禁止的三种转换，状态迁移由实体自身执行，应用服务不得直接修改状态字段。

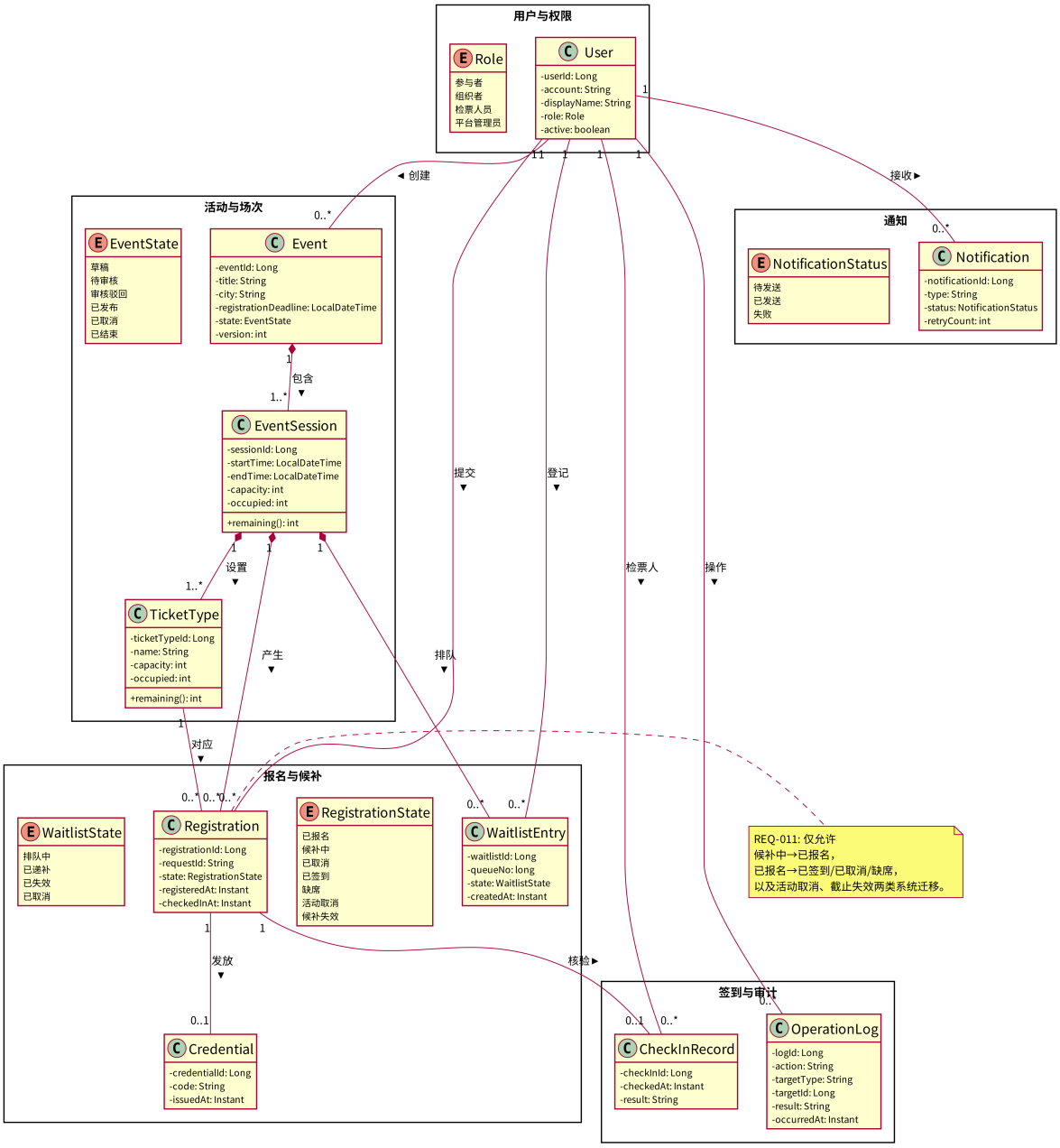


图 2-1 类图（领域模型）

2.1.2 服务类图

报名是最复杂的用例，图 2-2 单独画出围绕报名服务的协作对象。RegistrationService 只负责编排：先由 IdempotencyService 以 requestId 查重，再由 DeadlinePolicy 用 ServerClock 判定开放期，随后通过 TransactionManager 开启事务并锁定场次，最后委托 WaitlistManager 处理满额候补。把时钟、截止策略、授权、事务与候补分别做成可替换的协作对象，而不是塞进一个巨大的服务方法，一方面便于按 REQ-014、REQ-028 单独验证，另一方面让领域实体不依赖框架与运行环境。

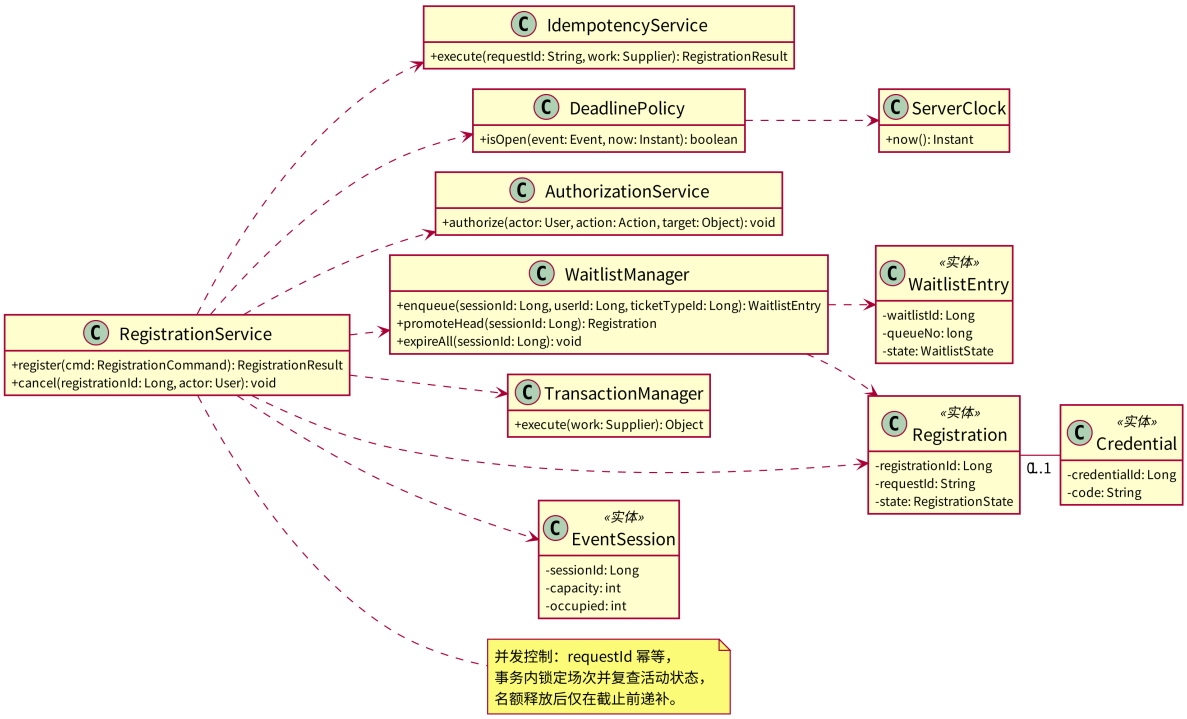


图 2-2 服务类图（报名服务协作）

2.1.3 组件图

图 2-3 采用四层结构：表现层按角色分为参与者端、组织者端、检票端与管理端四个页面组；应用服务层按用例划分活动、审核、报名、检票、统计导出与通知六个服务；领域层放置活动与场次模型、报名与候补模型、签到与审计模型以及业务策略；基础设施层提供关系数据库、事务管理器、服务器时钟、CSV 导出器与通知渠道。依赖只允许自上而下，页面不直接访问数据库，事务与时钟也不由页面或领域对象创建。这样设计的原因是：实验一中的截止判定（REQ-014）、事务一致性（REQ-028）与越权防护（REQ-030）都属于“写入路径上的统一关卡”，放在应用服务层与基础设施层才能一处生效、全局一致。

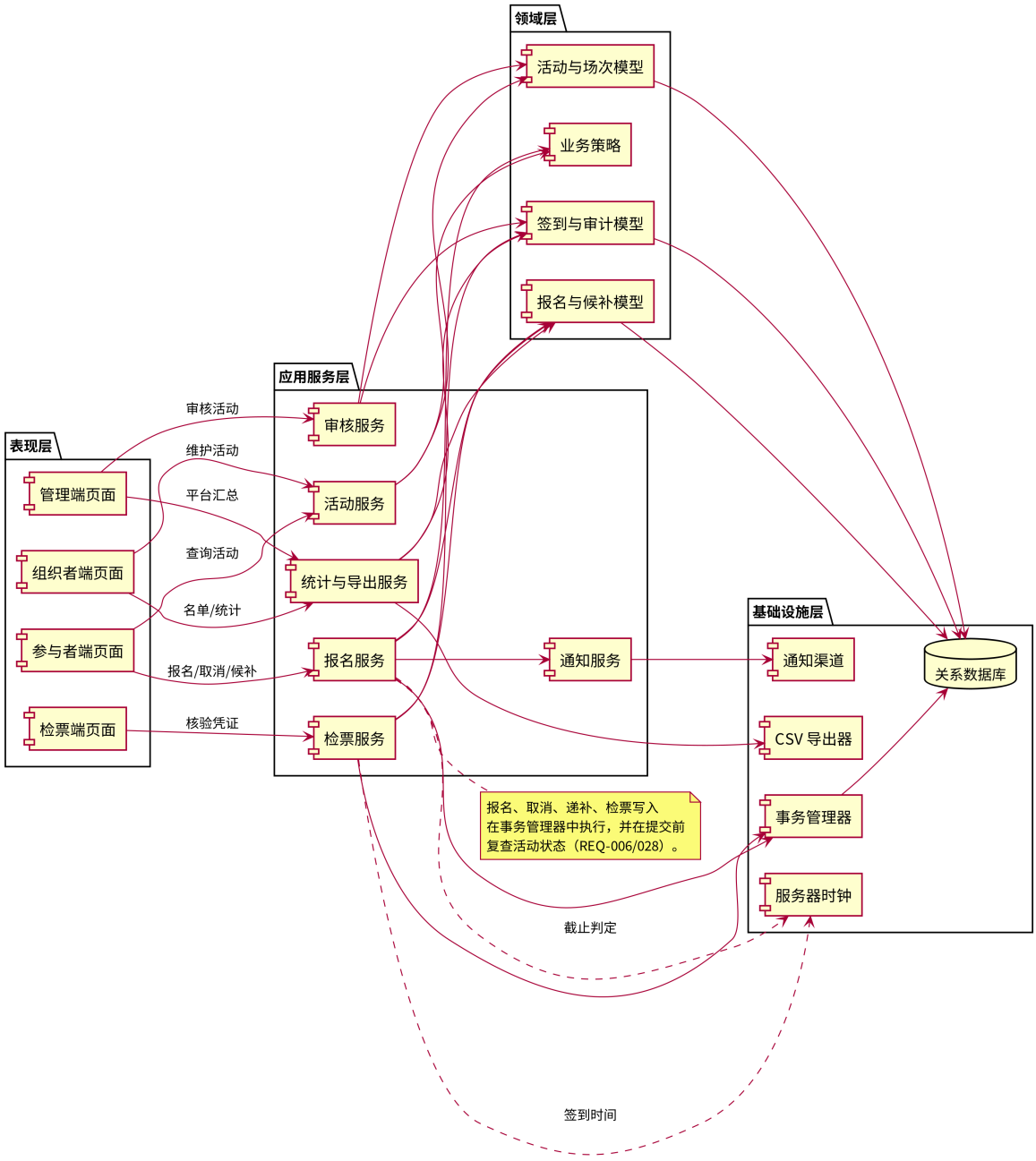


图 2-3 组件图（分层结构）

结构性图遵循同一原则：实体负责状态与不变量，服务负责用例编排，基础设施负责技术与事务。三类职责不混在一起，后续的动态图才能与结构图一一对应。

2.2 行为性图

2.2.1 用例图

图 2-4 以五类参与者组织 20 个用例。除参与者、组织者、检票人员与平台管理员外，把自动递补、候补失效与缺席标记归给“定时任务”这一系统参与者，避免把

后台规则画成人工操作。用例按浏览与报名、活动管理、检票与统计、通知与审计四组排列；报名资格校验、场次授权校验、审计日志记录用 include 表示必经步骤，自动递补在“满额且有候补”时用 extend 表示可选扩展。该图对应 REQ-001 的多角色隔离，也表明 REQ-030 的越权防护发生在服务端而不是页面。

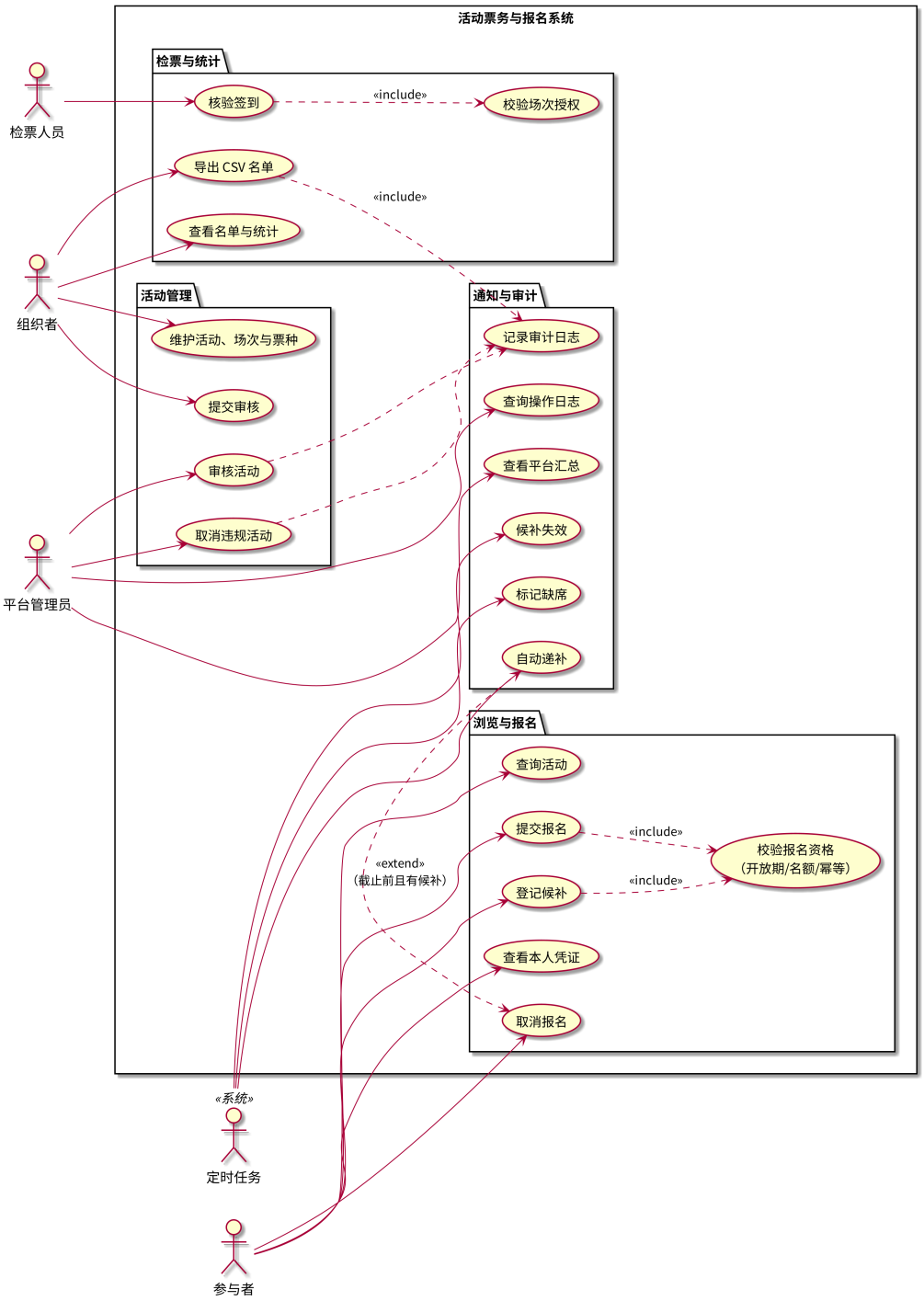


图 2-4 用例图

2.2.2 活动图

图 2-5 给出从报名到检票的活动流，重点是异常分支而不是顺利路径：先校验活动状态与截止时间，再检查requestId 是否重复；事务内锁定场次后才判断名额，满额则登记候补；检票阶段区分无权限、已签到与已取消；场次结束后统一把未签到的已报名记录标为缺席。把“拒绝原因”画进流程，是为了保证界面与接口的失败提示有明确来源，而不是由实现临时拼凑。

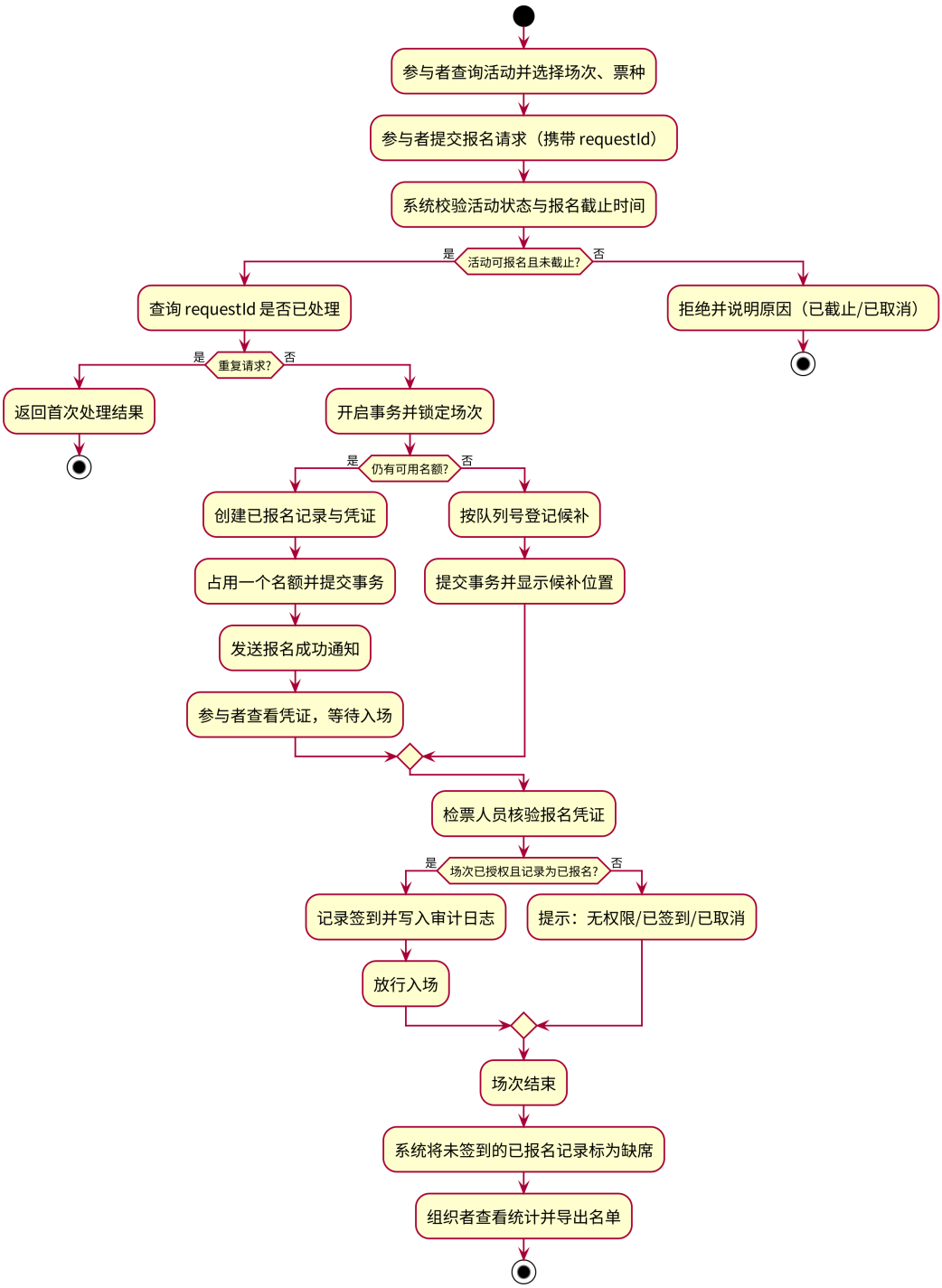


图 2-5 活动图（报名与检票主流程）

2.2.3 状态图

图 2-6 只画 REQ-011 明确允许的转换：候补中→已报名，已报名→已签到、已取消或缺席，以及候补中→候补失效、已报名→活动取消。已签到是终态，不因活动取消而回退，保证历史事实与日志不被抹除（REQ-018）；已取消、缺席与候补失效同样不再回到报名状态。图中注释列出三种禁止转换，并注明截止后取消不触发递补（REQ-015）。尚未确认的路径（例如取消后能否重新报名）没有画入，避免把猜测当成规则。

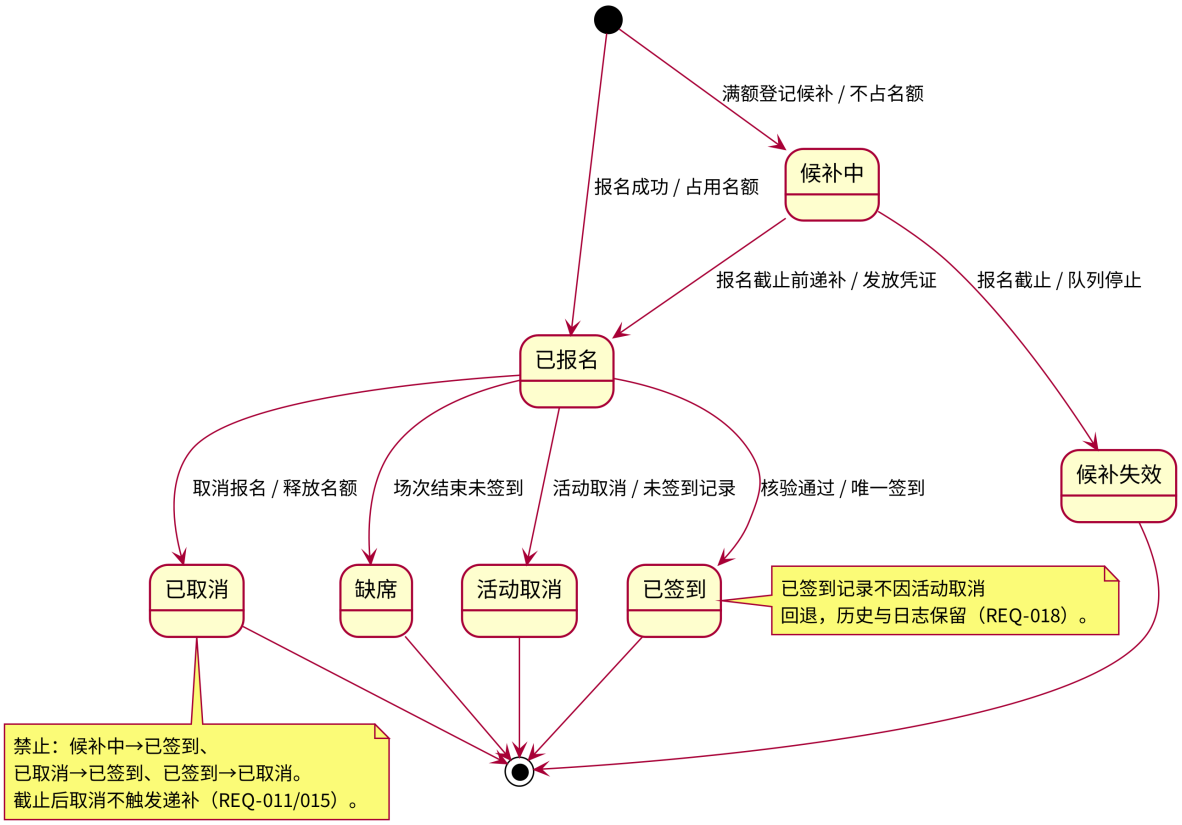


图 2-6 状态图（报名记录）

2.3 交互性图

2.3.1 时序图

图 2-7 分为两段。上半段是报名：界面提交命令后，服务先查 requestId 幂等记录，再用服务器时钟判断截止，然后在事务中锁定场次、复查活动状态并尝试占位；有名额则写入报名与凭证并提交，无名额则登记候补。下半段是取消与自动递补：取消释放名额后，在同一事务内取队首候补并转为已报名；通知在事务提交后发送，通知失败只记录失败并允许补发，不回滚报名（REQ-026）。该图把 REQ-012、REQ-013、REQ-014 与 REQ-028 的并发与失败边界落实到了具体调用顺序。

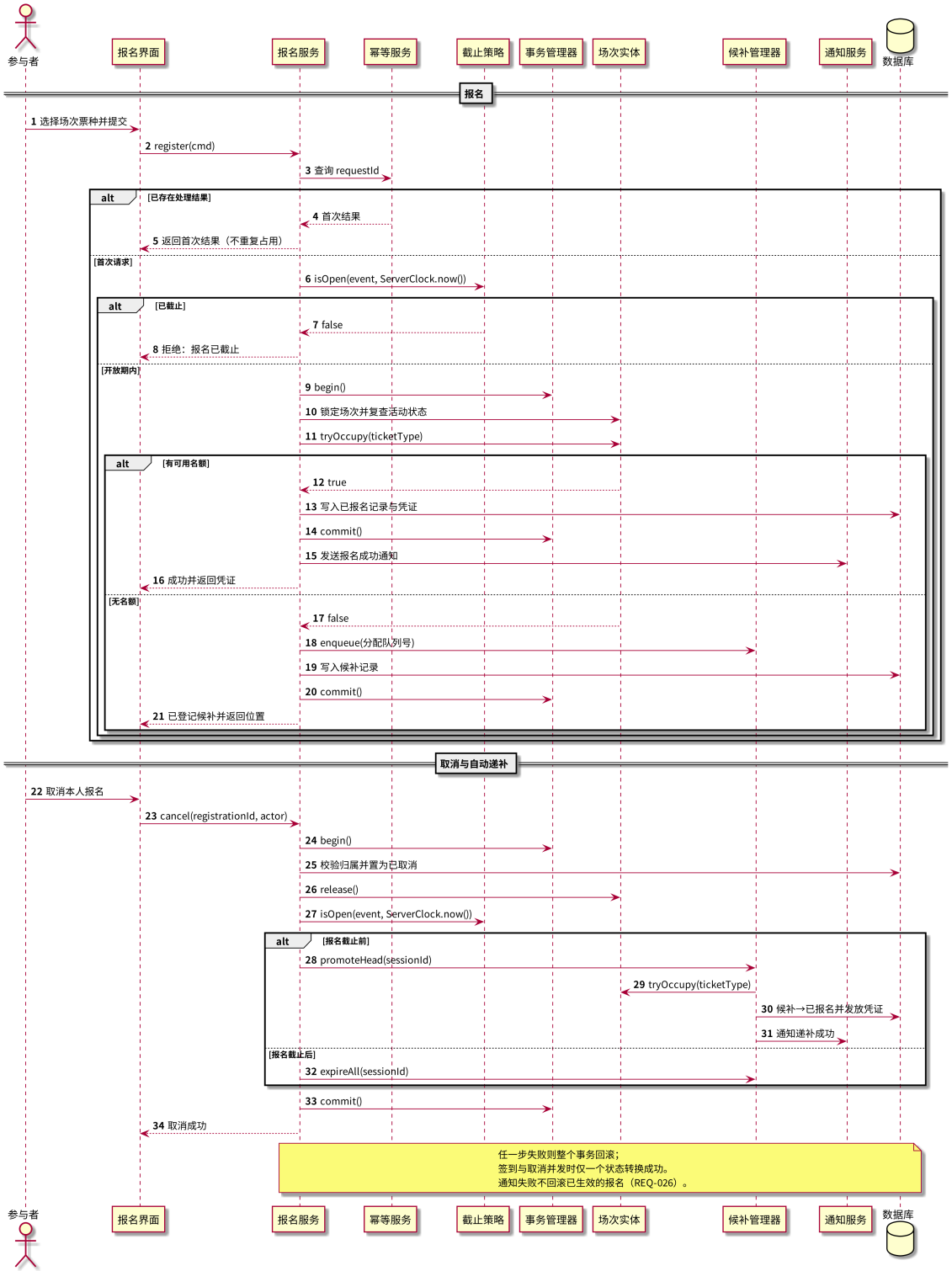


图 2-7 时序图（报名；取消与自动递补）

2.4 设计取舍

几张图之间共同的取舍可以归纳为六点：

- 事务边界放在应用服务，一次用例一个事务，领域对象不自行提交；

- 并发控制采用“事务 + 场次行锁 + 占用数条件更新”，保证最后一个名额只成功一次；
- 截止判定统一使用服务器时钟，客户端时间、旧页面缓存与修改参数都不参与；
- 状态迁移封装在实体内部，非法迁移抛出领域异常，并记录失败日志；
- 通知后置且不参与业务事务，失败可补发而不回滚已成功的报名；
- 审计作为横切能力，由应用服务在关键写入时统一记录操作人、时间、对象与结果。

2.5 架构小结

从结构到行为，本章的七张图回答同一个问题：需求中的约束分别落在哪里。角色与数据范围落在用例图和授权服务；活动生命周期落在 EventState 与审核服务；名额与并发落在场次实体、事务管理器与时序图的加锁顺序；截止与候补失效落在 DeadlinePolicy、ServerClock 与定时任务；签到与缺席落在检票服务与状态图。图与图之间保持同名、同状态、同职责，避免出现“类图有、时序图没有”的悬空设计。

同时也要说明边界：图中没有画入实验一尚未确认的规则，例如取消后能否重新报名、缺席后能否补签；这些位置只保留扩展点，待需求补充后再进入模型。这样做不是遗漏，而是遵守“未确认的转换一律不视为已确认规则”的审查结论。

三、UI 界面及功能描述

界面使用统一的 HTML/CSS 设计系统，主色为靛蓝，状态徽标语义固定：绿色表示已报名或成功，黄色表示候补或待审核，红色表示缺席或失败，灰色表示已取消或失效。所有页面按 1440 px 宽度设计，并突出“名额、截止时间、当前状态、下一步操作”四项信息。以下 7 个界面覆盖四类角色与主要异常状态。

活动发现页是参与者的入口。顶部提供关键词、城市、类别与时间四个可组合条件；卡片显示场次时间、报名截止、已报名与容量、剩余名额，满额活动直接提示“可登记候补”。默认只展示已发布且未结束的活动，空结果与默认列表由后端查询统一处理。该页对应 REQ-007，并为 REQ-008 的报名入口提供上下文。

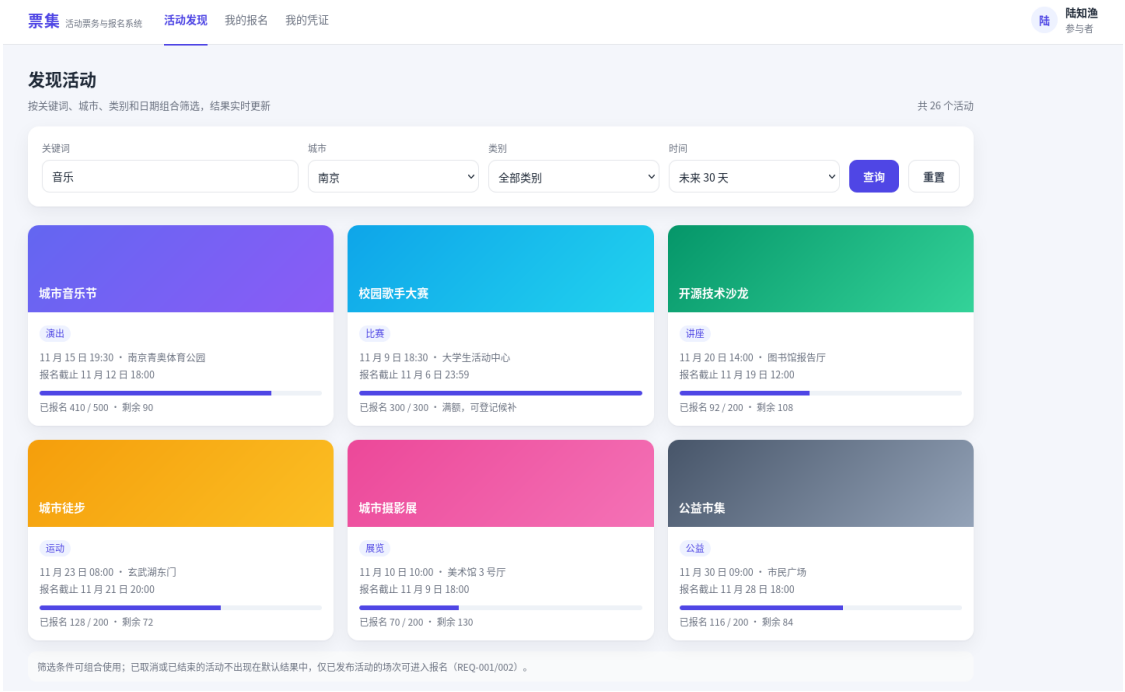


图 3-1 活动发现

详情页把场次与票种拆成两级选择，右侧固定显示当前选择的剩余名额、截止时间与两个动作：立即报名、加入候补。提示明确“名额在提交时由服务端重新校验”，对应最后名额并发（REQ-012）与截止限制（REQ-014）；“重复提交返回首次结果”对应幂等要求（REQ-013）；无名额时引导候补对应 REQ-010。

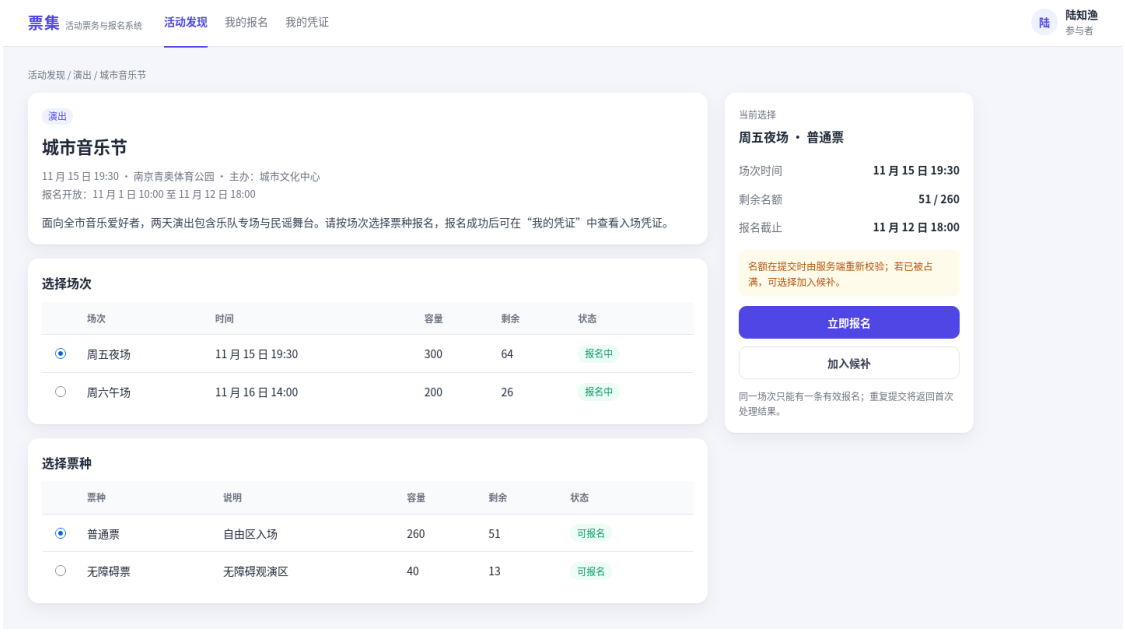


图 3-2 活动详情与报名

工作台以活动列表为主线，展示草稿、待审核、已发布与已取消四种状态，并提供提交审核入口；下方为场次与票种容量表，调整容量时提示不得低于当前占用数（REQ-004、REQ-021）。已发布活动的重要信息修改需要复审，已取消活动只能查看记录，不再接受报名或检票（REQ-006）。



图 3-3 组织者工作台

该页只显示当前用户的数据，列表覆盖已报名、候补中、已签到与已取消四种展示状态，操作按钮随状态变化：已签到不再提供取消，已取消只提示名额已释放。右

侧凭证卡显示凭证码、持票人、票种、场次与有效性，并说明核验后不可取消（REQ-011、REQ-018）。该页对应 REQ-008 与 REQ-009，也是 REQ-001 数据隔离的界面体现。

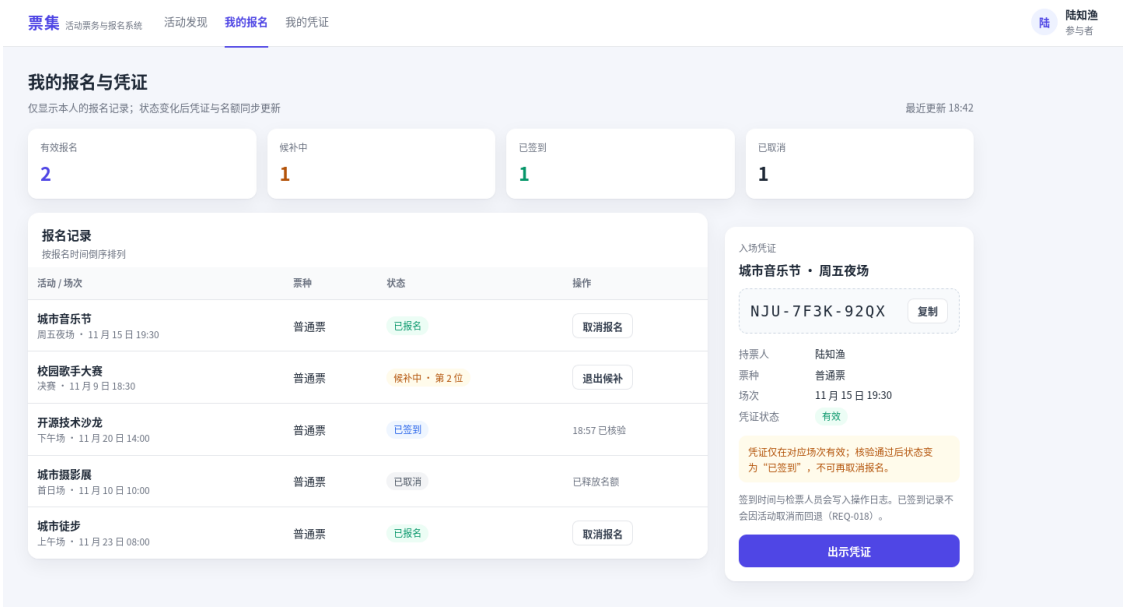


图 3-4 我的报名与凭证

名单页按场次展示已报名、候补、剩余名额与签到率，签到率严格按“已签到 ÷（已签到 + 缺席）”计算，并标注分母为零时的显示规则（REQ-022）。列表可按票种、状态、姓名或账号筛选并分页，右上角导出 CSV；底部说明导出中断会返回失败并记录日志，不把不完整文件标为成功（REQ-023）。



图 3-5 组织者名单与统计

核验页只列出被授权的场次，凭证码输入配合“核验并签到”完成一次核验。成功卡显示持票人、票种、场次、签到时间与检票人；失败区列出场次未授权、活动已取消、凭证不属于本场次、记录不是已报名与重复核验五类原因。右侧实时显示本场签到数与最近记录，重复核验只提示“已签到”而不重复计数（REQ-016、REQ-017）。



图 3-6 检票核验

平台管理页左侧是待审核活动，管理员可以直接通过并发布或驳回，也可查看详情；顶部统计待审核、已发布、今日报名与失败操作。右侧操作日志按时间倒序展示操作、操作人与结果，并提供成功与失败筛选；底部强调失败操作同样保留日志（REQ-025）。该页同时承载活动审核（REQ-003）、违规取消（REQ-006）与平台汇总（REQ-024）。



图 3-7 管理员审核与操作日志

四、大模型的使用

4.1 使用原则

本次实验把大模型定位为“初稿生成器”，而不是设计决策者。每次使用前先明确输入（需求编号、约束、验收标准）与期望产物（可渲染的 PlantUML 源码、可截图的 HTML）；生成后必须经过三步校验：源码能否渲染、图中元素能否在需求中找到依据、异常与边界场景是否被覆盖。四次主要使用及其修改结果记录在仓库的 design_log.md 中，并分别对应 Git 标签。

4.2 使用记录

AI-01 类图与组件图初稿

使用场景：把实验一 29 条已确认需求的摘要交给大模型，要求划分模块并生成类图与组件图的 PlantUML 源码。

生成结果：得到“用户管理、活动管理、报名管理”三个包、11 个类的类图，以及“前端—后端服务—数据库”三层组件图。

优点：主流程覆盖快，PlantUML 语法基本正确，包结构直观，能迅速搭起讨论框架。

缺点：界面类混入领域类图；容量只放在票种上，场次没有容量；报名状态用字符串、签到用布尔字段；缺少候补、凭证、幂等、截止、事务、审计与通知；组件图让页面直接依赖数据库。

修改优化后的结果：按需求逐条重建模型（REV-001）：删除界面类与登录实现细节，引入 Role、EventState、RegistrationState 等枚举；EventSession 与 TicketType 分别维护容量和占用并提供 remaining()；新增 Credential、WaitlistEntry、IdempotencyService、DeadlinePolicy、TransactionManager、ServerClock 等；组件图改为四层。结果见图 2-1 至图 2-3。

AI-02 用例图、活动图、状态图、时序图初稿

使用场景：要求模型依据初稿类图补齐动态图，覆盖用户操作流程。

生成结果：用例图只有 4 个参与者与 7 个用例；活动图是一条无分支的顺利路径；状态图只有“已报名、已签到、已取消”；时序图是“界面—服务—数据库”的直通调用。

优点：图形结构清楚，能作为讨论起点，节省了从零绘制的时间。

缺点：缺少检票人员与定时任务两个参与者；活动图没有满额、候补、截止、取消、缺席分支；状态图允许“已取消→已报名”“已签到→已取消”，与 REQ-011 明确冲突；时序图没有服务器时间、事务、行锁与幂等键，最后名额会超卖，与 REQ-012、REQ-013、REQ-028 冲突。

修改优化后的结果：以异常与边界场景为准重画（REV-002）：增加两个参与者、20 个用例与 include/extend；活动图加入截止、幂等、事务与锁、满额转候补、检票失败与缺席标记；状态图只保留允许转换并写出禁止转换；时序图改为报名与“取消—递补”两段，补上 ServerClock、DeadlinePolicy、TransactionManager 与通知失败边界。结果见图 2-4 至图 2-7。

AI-03 UI 界面初稿

使用场景：让大模型生成参与者与组织者界面的 HTML/CSS 原型，供后续截图。

生成结果：3 个界面：活动发现、活动详情与报名、组织者工作台；统一了主色与卡片样式。

优点：布局与视觉基线可用，信息层级清楚，后续改造成本低。

缺点：只覆盖两类角色，缺检票端、管理员端与“我的报名/凭证”；没有空态、错误态、候补位置、导出失败提示等反馈；状态徽标没有统一语义，容量与名额信息分散。

修改优化后的结果：补齐 7 个界面并抽取公共组件（REV-003）：统计卡、状态徽标、双栏、结果提示、凭证码与分页；固定灰、绿、黄、红四种徽标语义；为满额候补、服务端重校验、重复提交、导出失败、重复核验、已签到不可取消与失败日志等边界加上可见反馈；用无头浏览器逐页截图。结果见图 3-1 至图 3-7。

AI-04 实验报告初稿

使用场景：按实验要求让大模型生成报告大纲与章节文字。

生成结果：得到功能罗列、UML 图说明和 AI 使用章节的初稿。

优点：章节齐全，术语基本正确，可快速形成报告骨架。

缺点：内容泛化，未把设计与实验一的需求编号对应；把“尚待确认”的规则写成已实现；图表说明与图本身不一致；AI 使用记录只有结论，没有生成结果与修改过程。

修改优化后的结果：按“模块—需求映射”重写，逐张图说明设计原因；把 AI 使用改写为“场景、生成结果、优缺点、修改结果”四段式；对未确认规则只保留扩展点，不擅自补定（REV-004）。

4.3 工作流反思：能否只做检查校对

四次使用呈现相同规律：模型擅长在已有约束下铺开结构、生成语法正确的产物并提供备选方案；不擅长主动发现需求之间的冲突、判断哪些规则尚未确认，也难以保持多张图之间的一致性。它的错误往往是“看起来合理但缺少边界”，例如状态图少一个终态、时序图少一把锁，表面完整，实际无法通过验收。

可以把工作流压缩为“人定约束—机出初稿—自动校验—人工审查—版本留痕”五步：一，人工准备需求基线、术语表与禁止项；二，把需求编号与验收标准随提示词交给模型，要求输出可渲染的源码；三，用 PlantUML 渲染、HTML 截图、PDF 编译等自动步骤淘汰语法与排版错误；四，人工逐条对照需求审查，重点看异常分支、状态迁移与跨图一致性，并把修改写成 REV 记录；五，用 Git 标签区分 AI 初稿与每轮人工修改，便于回溯。

但“只做检查校对”目前仍不现实：模型无法替人决定哪些需求尚未确认、哪些取舍可以接受，也无法对界面审美和业务优先级负责。更现实的分工是让模型承担约六到七成的草案与重复劳动，人保留需求判断、架构取舍与验收责任。当需求足够形式化、验收标准可自动执行时，检查校对的比例可以继续提高。

4.4 对大模型辅助软件设计的思考

从本次体验看，大模型适合四类工作：一是把散落的需求整理成候选结构；二是生成可执行的模型与原型，让讨论有具体对象；三是做一致性检查，例如列出图中缺少的状态或未覆盖的异常；四是快速产出多套界面方案供比较。要让结果可靠，提示词必须携带约束与验收标准，产物必须可验证，修改必须留痕。把它当作“会犯错的初级设计助手”，而不是“自动完成设计的工具”，是目前较稳妥的用法。

五、Git 版本记录

实验一与实验二在同一仓库的 lab1、lab2 两个分支上进行。实验二从 AI 初稿标签开始，每次人工修改单独提交并打注释标签，标签之间可以直接 diff 出人工改动；标签数量为 5 个，满足实验要求。

标签	提交	内容
lab1-ai-draft	1f11835	实验一：AI 需求初稿
lab1-final	11b0894	实验一：需求规格说明书定稿
lab2-ai-draft	43680a9	实验二：AI 生成的 UML 与 UI 初稿
lab2-rev-1	da7f08d	人工重建领域模型与分层组件图
lab2-rev-2	102dfe6	人工补齐动态行为、异常与并发路径
lab2-rev-3	a6d2e78	人工补齐四类角色 UI、状态与反馈
lab2-final	本次提交	报告定稿、界面截图与打包

表 5-1 Git 标签与内容

使用 git log --oneline --decorate 可以查看标签位置，使用 git diff lab2-ai-draft..lab2-final 可以查看从 AI 初稿到最终提交的全部人工改动。仓库地址为 https://github.com/kkksc114/Software_Engineering_Lab ，分支为 lab2。

5.1 提交与标签约定

提交信息统一使用“类型: 内容”的格式，类型包括 docs（文档与报告）、design（UML 与设计）、ui（界面）、chore（打包与杂项）。标签使用注释标签（annotated tag），标签说明写明该阶段完成的人工修改；AI 生成的初稿与人工修改不混在同一次提交中，保证审查时能单独查看每一轮的改动。

六、总结

本次实验完成了从需求到设计模型的转换：8 个功能模块与 29 条已确认需求逐条对应，7 张 UML 图覆盖结构、行为与交互三类，7 个 UI 界面覆盖四类角色与主要异常状态；AI 的每次使用都有场景、结果、优缺点与修改记录，Git 上保留 5 个可追溯标签。

仍有明确的边界：对实验一中尚未确认的规则（如截止时刻的等号口径、缺席后能否补签、取消后能否重新报名），设计中只保留扩展点，没有擅自补定；UI 为静态原型，未接入真实服务；REQ-029 的性能指标需要在约定的软硬件环境中实测，本报告不宣称已完成性能验收。

最大的收获是：UML 的价值不在画得全，而在每个元素都能回答“对应哪条需求、为什么这样设计”；AI 能显著加快初稿，但边界条件、状态一致性与架构取舍仍需人来负责。

附录 A 提交文件结构

```
实验2_241880173_陆知渔/
├── 实验报告.pdf
├── UML/
│   ├── UML类图.puml / .png
│   ├── UML服务类图.puml / .png
│   ├── UML组件图.puml / .png
│   ├── UML用例图.puml / .png
│   ├── UML活动图.puml / .png
│   ├── UML状态图.puml / .png
│   └── UML时序图.puml / .png
└── UI界面/
    ├── 01-活动发现.png ... 07-管理员审核与日志.png
    └── 对应 HTML 源文件与 style.css
```

附录 B 图表生成方式

全部 UML 图使用 PlantUML 编写源码并渲染为 PNG（命令：plantuml -tpng -Sdpi=300 UML/*.puml，并设置 PLANTUML_LIMIT_SIZE=16384 避免大图被裁切），中文使用 Noto Sans CJK SC 字体。UI 页面使用 HTML/CSS 编写，通过 Firefox 无头模式按 1440 px 宽度截图。实验报告使用 Python 与 ReportLab 排版，正文为宋体，标题为方正小标宋。

附录 C 需求追溯

每张 UML 图与每个界面均在正文中标注了对应需求编号；表 1-2 给出模块级映射。实验一原始需求见仓库根目录的 requirements.md，其中 REQ-027 已合并至 REQ-023，当前有效需求为 29 条。